

DOCUMENT NO. SRS-01 · REV 0 (DRAFT)

Software / System Requirement Specification

"Akıllı Sayaç" Smart-Meter Gateway

DKM-440 → ESP32 → MQTT (Cellular APN) → PC Head-End

Project	OSOS-style smart-meter simulation (MQTT emulator board)
Document ID	SRS-01
Date	2026-07-13
Revision	0 — Draft for approval
Author	Tuğrul Altınüzengi (with Claude)
Source repository	D:\Dekstop\Coding\modbus_junk\

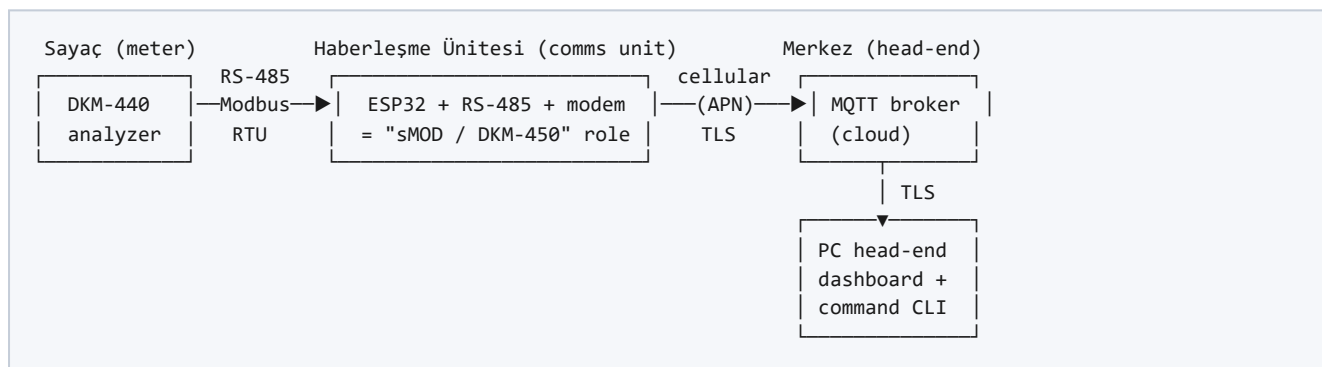
Contents

1. Introduction — purpose, scope, definitions, references
2. Overall Description — constraints, assumptions, bill of materials, wiring
3. Functional Requirements — gateway firmware (FW), broker (BR), head-end (HE)
4. Non-Functional Requirements (NF)
5. Interface Specification — MQTT topics and payload schemas
6. Phasing & Acceptance Criteria
7. Risks

1 Introduction

1.1 Purpose

Simulate the working principle of a Turkish **Akıllı Sayaç** (smart meter) reading chain (**OSOS** — Otomatik Sayaç Okuma Sistemi / **MASS** — Milli Akıllı Sayaç Sistemleri):



The DKM-440 power analyzer plays the meter. The ESP32 plays the communication unit (real-world equivalents: DATAKOM DKM-450 and İKOM sMOD-MASS modems — see [modbus/450_TAN.pdf](#) and [modbus/docs/IKOM_SIMULATION_MAPPING.md](#)). The PC plays the central OSOS software. Commands travel PC → broker → ESP32 → DKM-440; telemetry travels the reverse path.

1.2 Scope

In scope

- ESP32 firmware (PlatformIO, Arduino framework) that polls the DKM-440 over Modbus RTU and speaks MQTT over TLS.
- Two interchangeable network bearers: Wi-Fi (bench) and cellular via APN (field), selected at compile time. Same MQTT logic on both.
- A cloud MQTT broker (free tier) used **from day 1** for both bearers.
- MQTT topic schema + JSON payload schema (Section 5).
- PC head-end: extend the existing FastAPI webapp ([modbus/webapp/](#)) with an `mqtt` ingest source, plus a small command publisher (CLI + web buttons).

- Commands: on-demand read, reporting-interval change, ESP32 relay toggle, guarded DKM-440 register writes.

Out of scope

- Real OSOS protocol/encryption compliance (TEDAŞ MLZ/2019-064.A). The *principle* is imitated, not the wire protocol.
- Private APN / static-IP M2M SIM (documented as a future option only).
- Energy (kWh) registers — not part of the confirmed map yet.
- The neural-network project (separate document SRS-02).

1.3 Definitions

Term	Meaning
OSOS / MASS	Turkish national automatic / smart meter reading systems
APN	Access Point Name — the gateway a SIM uses to obtain an IP address from the operator. Consumer APNs give a private, carrier-NAT'd IP : outbound connections only
MQTT	Publish/subscribe messaging protocol. Both ends dial out to a broker, so carrier NAT is never a problem
Broker	The MQTT middleman server both the ESP32 and the PC connect to
QoS	MQTT delivery guarantee: 0 = at most once, 1 = at least once
LWT	Last Will and Testament — message the broker publishes on behalf of a client that dies unexpectedly (how the head-end knows the meter went dark)
Retained message	Broker stores the last message on a topic and replays it to new subscribers (used for status)
FC03	Modbus function code 3, Read Holding Registers
TEDAŞ float map	The CONFIRMED register map: IEEE-754 float32, 2 registers per value, big-endian word order, wire address = documented 4xxxx – 40000. Source of truth: <code>dkm_440_analyze/map.py</code>

1.4 References

- `dkm_440_analyze/map.py` — **confirmed** register map + polite polling code (to be reused)
- `modbus/c9f61b48-*.pdf` — TEDAŞ MLZ/2017-063 specification, Annex EK-A Modbus map
- `modbus/450_TAN.pdf` — DKM-450 communication unit (the device being imitated)
- `modbus/docs/IMPLEMENTATION.md` — earlier staged plan (wiring reused; ThingsBoard part superseded by this specification)
- `modbus/docs/DKM440_CHEATSHEET.md` — device quirks
- `modbus/webapp/` — existing FastAPI head-end to extend

2 Overall Description

2.1 Constraints (hard, from prior reverse-engineering)

ID	Constraint
C-1	Serial: 19200 baud, 8N1, slave ID 1 . One master at a time on the bus
C-2	$\geq$ 30 ms gap between consecutive Modbus reads; the DKM-440 tarpits back-to-back masters. No library auto-retry; one paced retry per value
C-3	Read start addresses must be even-aligned
C-4	Never read the command block 16385–16406 (write-only; reads hang)
C-5	After a Modbus timeout, flush/reset the serial link (a stale half-frame corrupts everything after)
C-6	Floats decode as FLOAT32, big-endian word order (high word at lower address)
C-7	Broker requires TLS (port 8883) + username/password on the free tier
C-8	Cellular modem burst current up to ~ 2 A — needs its own buck converter + bulk capacitor; never the ESP32 3V3 pin

2.2 Assumptions

- Hardware arrives 2026-07-14 (ESP32 devkit, RS-485 transceiver, LTE modem + data SIM).
- Modem model undecided → firmware must be **modem-agnostic via TinyGSM** (`#define TINY_GSM_MODEM_A7670` etc., one line per model). Recommended purchase: **SIMCom A7670E** (same family as the real IKOM modem).
- A consumer data SIM with a standard internet APN (e.g. `internet / mgb`) is sufficient — MQTT needs outbound connectivity only.
- The PC keeps its existing role: it may still run `dkm_440_analyze/server.py` for direct-serial debugging, but **not simultaneously** with the ESP32 on the bus (C-1).

2.3 Bill of Materials

Item	Notes
ESP32 devkit (WROOM/WROVER)	any 30/38-pin devkit
RS-485 transceiver	MAX3485 (3.3 V) preferred; MAX485 (5 V) acceptable — RO output reads fine on the ESP32; keep DI/DE/RE at 3.3 V logic
LTE modem + antenna + data SIM	A7670E recommended; SIM7600E acceptable; SIM800L discouraged (2G sunset)
120 Ω resistor	RS-485 termination at cable end
Relay or LED module	the “breaker” output

5 V $\geq$ 3 A supply + buck to 3.8–4.2 V	modem power (C-8)
Dupont wires; common GND everywhere	RS-485 needs the GND reference

2.4 Wiring (retained from IMPLEMENTATION.md)

- ESP32 **UART2**: GPIO16 = RX $\leftarrow$ MAX485 RO; GPIO17 = TX $\rightarrow$ MAX485 DI
- MAX485 **DE + RE tied together** $\rightarrow$ GPIO4 (HIGH only while transmitting)
- MAX485 A/B $\rightarrow$ DKM-440 RS-485 A/B, common GND, 120 Ω across A/B
- Relay/LED $\rightarrow$ GPIO26 (configurable)
- Modem on **UART1** (e.g. GPIO25 RX / GPIO27 TX, configurable), powered separately

3 Functional Requirements

3.1 Gateway Firmware (ESP32) — FW-*

ID	Requirement
FW-1	Poll the confirmed TEDAŞ float map (all ~23 values from <code>map.py</code> : L1/L2/L3/N voltage, currents, frequency, PF, $\cos \varphi$, $\tan \varphi$, supply voltage) every report interval (default 5 s, range 1 s–15 min) respecting C-2/C-3/C-6
FW-2	Publish each poll as one JSON telemetry message (§5.2) with QoS 1
FW-3	Publish retained status (<code>online</code>) on connect; register LWT so the broker publishes <code>offline</code> if the gateway dies (§5.2)
FW-4	Subscribe to the command topics (§5.1) and execute: <code>read_now</code> (immediate poll + publish), <code>set_interval</code> (persist to NVS), <code>relay</code> (GPIO on/off), <code>dkm_write</code> (single/multi register write)
FW-5	Every command gets an ack message echoing the command <code>id</code> with <code>ok/error</code> + detail (§5.3)
FW-6	<code>dkm_write</code> is feature-flagged (<code>ALLOW_DKM_WRITE</code> , default off) and restricted to an allowlist of addresses (initially the 16385–16406 command block). The command block is unverified — first enable only in a supervised bench session
FW-7	Time sync via NTP (Wi-Fi) or modem network clock (cellular); every message carries an ISO-8601 UTC timestamp + monotonic <code>seq</code> number
FW-8	Store-and-forward : while the broker is unreachable, buffer up to 512 readings in RAM (ring buffer); flush oldest-first on reconnect, flagged <code>"buffered": true</code> . (Mirrors what real OSOS modems do during outages)
FW-9	Network bearer selected at compile time: <code>BEARER_WIFI</code> or <code>BEARER_CELL</code> (TinyGSM + configurable APN/user/pass). All MQTT logic identical
FW-10	Reconnect with exponential backoff (1 s → 60 s cap) for both the network and the broker; hardware watchdog resets the board if the main loop stalls > 30 s
FW-11	All secrets/IDs (Wi-Fi, APN, broker host/user/pass, device id) in one <code>config.h</code> excluded from git (<code>config.example.h</code> committed)
FW-12	If the DKM-440 stops answering, keep publishing telemetry with per-register <code>null</code> values and raise an <code>event</code> message — link loss is data, not silence

3.2 Broker — BR-*

ID	Requirement
BR-1	Cloud free tier (EMQX Serverless or HiveMQ Cloud), TLS on port 8883
BR-2	Two credentials: <code>gateway</code> (publish telemetry/status/ack, subscribe cmd) and <code>headend</code> (the reverse). No anonymous access

BR-3	Broker hostname/credentials documented in a private <code>BROKER.md</code> excluded from git
------	--

3.3 PC Head-End — HE-*

ID	Requirement
HE-1	New <code>mqtt</code> source in <code>webapp/ingest.py</code> (alongside <code>sim / modbus / push</code>) using <code>paho-mqtt</code> : subscribes to <code>telemetry/status/ack/event</code> and funnels into the existing <code>log → alarm → broadcast</code> pipeline unchanged
HE-2	Existing dashboard shows the MQTT-sourced live values; connection LED driven by the retained status topic
HE-3	<code>webapp/commands.py</code> publishes commands to MQTT instead of writing serial directly when <code>source = mqtt</code> ; the UI shows the matching ack (by <code>id</code>) or a timeout after 10 s
HE-4	A standalone CLI <code>tools/meter_cmd.py</code> can send any command and print the ack — usable without the webapp for debugging
HE-5	All readings persisted in the existing SQLite store (<code>store.py</code>)

4 Non-Functional Requirements

ID	Requirement
NF-1	End-to-end command latency (PC → ack received) < 2 s on Wi-Fi, < 5 s on LTE
NF-2	Telemetry payload ≤ 1 KB (cellular data budget: default settings < 30 MB/month)
NF-3	Gateway survives: broker restart, SIM data loss, DKM power-off, ESP32 brownout — all without reflash (FW-8/10/12 verify this)
NF-4	TLS everywhere; no credentials in git; broker ACL per BR-2
NF-5	Firmware code teaching-oriented: each module ≤ ~300 lines, header comment explaining its role in the OSOS analogy
NF-6	Console output ASCII-only (Turkish Windows cp1254 console)

5 Interface Specification

5.1 MQTT Topics ({gw} = device id, default dkm440-gw1)

Topic	Direction (gateway)	Retained	Payload
osos/{gw}/telemetry	publish	no	\$5.2 telemetry
osos/{gw}/status	publish (+ LWT)	yes	\$5.2 status
osos/{gw}/event	publish	no	link-loss, boot, buffer-flush notices
osos/{gw}/cmd	subscribe	no	\$5.3 command
osos/{gw}/ack	publish	no	\$5.3 ack

5.2 Telemetry / Status Payloads

```
// telemetry
{ "ts": "2026-07-14T09:30:05Z", "seq": 1041, "buffered": false,
  "values": { "MainBus_Voltage_L1": 231.4, "MainBus_Current_L1": 3.02, "...": 0 },
  "meter_ok": true }

// status (retained)
{ "state": "online", "fw": "0.1.0", "bearer": "wifi|cell", "ip": "10.x.x.x", "rssi": -67 }
```

Value keys = the exact key names from dkm_440_analyze/map.py (single source of truth; the firmware map is **generated** from it, not hand-copied).

5.3 Command / Ack Payloads

```
// cmd                                // ack
{ "id": "c-0007",                      { "id": "c-0007",
  "action": "read_now" }                "ok": true, "detail": "published seq 1042" }

{ "id": "c-0008", "action": "set_interval", "seconds": 60 }
{ "id": "c-0009", "action": "relay", "on": true }
{ "id": "c-0010", "action": "dkm_write", "addr": 16387, "values": [1] }
```

Unknown action or disallowed addr → "ok": false ack, never silence.

6 Phasing & Acceptance Criteria

Phase 0 — bench sanity (no new code). DKM-440 readable via dkm_440_analyze/server.py on COM13. ✓ already true.

Phase A — Wi-Fi + cloud broker.

ID	Criterion
----	-----------

A1	<code>mosquitto_sub</code> on the PC shows telemetry every 5 s with plausible voltages (match <code>server.py</code> values ± 1 V)
A2	Pulling the gateway's power flips retained status to <code>offline</code> (LWT) within the keepalive (~30 s)
A3	All four commands round-trip with correct acks from <code>tools/meter_cmd.py</code> ; <code>set_interval 60</code> visibly changes cadence and survives reboot
A4	Unplugging the RS-485 cable → <code>values</code> null + <code>event</code> published (FW-12); replugging recovers without reboot

Phase B — cellular APN.

ID	Criterion
B1	Same firmware rebuilt with <code>BEARER_CELL</code> , SIM + APN configured → telemetry arrives at the same broker with Wi-Fi disabled
B2	Commands from the PC reach the gateway over carrier NAT (< 5 s ack)
B3	Pull the antenna 2 min → readings buffered, then flushed on reconnect in order, marked <code>buffered: true</code> (FW-8)

Phase C — head-end integration.

ID	Criterion
C1	Webapp dashboard live-updates from the <code>mqtt</code> source; command buttons show acks
C2	24 h soak: zero manual interventions; gaps in SQLite only where events explain them (NF-3)

7 Risks

Risk	Mitigation
DKM write block semantics unverified	FW-6 flag default-off; first test supervised with only register 16387
Modem model unknown until purchase	TinyGSM abstraction; only a <code>#define</code> + UART pins change
2G sunset if SIM800L gets bought anyway	Specification recommends A76xx; SIM800L accepted only as fallback
Free-tier broker limits (connections/messages)	Both fit easily (< 1 msg/s); documented in <code>BROKER.md</code>
MAX485 is a 5 V part	Prefer MAX3485; otherwise verify RO high level; power at 5 V with a 3.3 V logic-tolerance check